UNIFEOB

CENTRO UNIVERSITÁRIO DA FUNDAÇÃO DE ENSINO OCTÁVIO BASTOS
ESCOLA DE NEGÓCIOS

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
CIÊNCIAS DA COMPUTAÇÃO

Tutorial completo: SOUFER Tools na AWS, com otimização e monitoramento

Entrega Prof. Rodrigo Marudi 03/11/2026 · versão de 10/11/2026

ESTUDANTES:

25000151 - Guilherme P. da Rosa Santi
25001176 - Henrique de Oliveira Molinari
25000019 - João Augusto de Freitas
25000795 - Kauan Leander Leandrini

São João da Boa Vista–SP
2026

Versão de 10/11/2026 (atividade do Prof. Rodrigo Marudi, 03/11/2026). Este guia refaz, na ordem certa e já com as correções, tudo o que fizemos: colocar o sistema no ar numa EC2 do AWS Learner Lab, medir o desempenho, monitorar, otimizar, endurecer a segurança, fazer backup e reunir as evidências. O relatório que resultou deste trabalho é `entrega-marudi-10-11-2026.pdf`, na mesma pasta.

Como usar este tutorial

- **Onde cada comando roda.** Blocos `bash` rodam no terminal da **EC2** (EC2 Instance Connect). Blocos `powershell` rodam no **seu computador**, na pasta do projeto. Passos feitos no navegador dizem "Console da AWS".
- **Pronto quando.** Cada passo termina com o que você deve ver. Se não viu, não avance.
- **Correção.** Os quadros "Correção" marcam os pontos em que nós erramos na primeira vez e o que fazer para não repetir.
- **Extra (não feito por nós).** Itens marcados assim são recomendados, mas não foram executados na nossa entrega. O relatório não os afirma.
- **Evidências.** Em cada passo há uma linha "Print". Tire o print com a data e a hora visíveis (relógio do Windows na tela) e guarde em `docs/entregas-pi/computacao-em-nuvem-rodrigo-marudi/prints-aws/`, com o número do passo no nome (por exemplo `4.8 Alarme de disponibilidade da API (1).png`).
- **Horários.** O console da AWS, o CloudWatch e o servidor usam UTC. Brasília é UTC−3 (22:02 UTC são 19:02).
- **Sessão do laboratório.** O Learner Lab dura cerca de 4 horas. Se acabar, clique em *Start Lab* de novo e inicie a instância em EC2 > Instâncias > Estado da instância > Iniciar. Com o IP elástico o endereço não muda, e o PM2 e o Nginx sobem sozinhos. As métricas do CloudWatch só existem enquanto a instância roda.

As 12 regras que evitam os nossos erros

1. Crie a instância por **Executar instâncias**, não por "Criar modelo de execução".
2. Escolha **Ubuntu** (22.04 ou 24.04), não Amazon Linux.
3. Use disco de **20 GiB**. O nosso foi de 8 GiB e chegou a 61% de uso.
4. Crie o **IP elástico** antes de qualquer teste.
5. Use senha de banco só com letras e números e fora de "root", "postgres", "admin", "123456" e "password" (a API recusa).
6. Cole os comandos em **blocos pequenos**: o terminal do navegador embaralha colagens longas. Se fechar a aba, rode de novo o `export` das variáveis.
7. Confira a variável `CORS_ORIGIN` depois de criar o `.env`. **Nunca** deixe o texto de exemplo (`IP_NOVO`, `SEU_IP`).
8. **Não deixe as contas do seed** (senha pública `123456`) ativas. Crie o seu administrador e inative as do seed.

9. **Confirme o e-mail da assinatura do SNS antes** de criar o alarme.
10. No teste do alarme, rode **um comando por vez** e espere. Parar e religar a API em sequência não dispara nada.
11. Meça o "antes" **antes** de mexer no Nginx, no PM2 e no PostgreSQL.
12. Nunca mostre em print o conteúdo do `.env`, o `JWT_SECRET`, a senha do banco ou um link de convite que ainda não foi usado.

Mapa do tutorial

Parte	O que faz
1	Criar a EC2, o IP elástico e conferir o firewall
2	Instalar e colocar o sistema no ar (com Nginx básico)
3	Medir a linha de base ("antes")
4	Monitoramento: CloudWatch, logs, alarme e painel
5	Otimização: Nginx, PM2 e PostgreSQL, e medir o "depois"
6	Segurança: CORS e cabeçalhos, `.env`, firewall, fail2ban, contas do seed, dependências
7	Backup, restauração testada e snapshot
8	Extras recomendados (HTTPS, teste de reinício, alarmes de recursos)
9	Evidências e relatório
10	Solução de problemas

Parte 1. Criar a infraestrutura

1.1 Ligar o laboratório

1. Na página do curso, clique em **Start Lab** e espere a bolinha ao lado de **AWS** ficar verde.
2. Clique em **AWS** para abrir o console. Confirme a região **N. Virginia (us-east-1)** no canto superior direito.

1.2 Executar a instância

Console da AWS > **EC2** > menu lateral **Instâncias** > botão laranja **Executar instâncias**.

Campo	Valor
Nome	`soufer-tools`
Sistema (AMI)	Ubuntu Server 22.04 ou 24.04 LTS, 64 bits (x86)
Tipo de instância	`t3.medium` (2 vCPU, 4 GiB)
Par de chaves	`vockey`
Rede > Editar	Permitir tráfego SSH e Permitir tráfego HTTP da internet

Campo	Valor
Armazenamento	**20 GiB gp3**

Clique em **Executar instância** e espere o estado ficar **Executando** com as verificações aprovadas.

> **Correção.** Se o título da página for "Criar modelo de execução" (Launch Template), cancele: é outra tela (Auto Scaling). Não deixe a AMI padrão (Amazon Linux): os comandos deste guia são do Ubuntu. E aumente o disco: com 8 GiB (o padrão do nosso volume) o uso chegou a 61%.

Pronto quando: a instância `soufer-tools` aparece como Executando e 2/2 (ou 3/3) verificações aprovadas.

1.3 Conferir o firewall (grupo de segurança)

1. Marque a instância, aba **Segurança**, e clique no link do grupo (`sg-...`, nome `launch-wizard-1`).
2. Em **Regras de entrada** devem existir só duas linhas: **SSH (22)** e **HTTP (80)**, origem `0.0.0.0/0`.
3. Se faltar o HTTP: Editar regras de entrada > Adicionar regra > Tipo HTTP > Qualquer local-IPv4 > Salvar.

O grupo da aba Segurança da instância deve ser o mesmo que você editou. Se aparecer o `default`, troque em Ações > Segurança > Alterar grupos de segurança. **Não** abra a porta 5432 (banco).

Print: `3.3 Grupo de segurança (firewall da AWS).png` (regras de entrada com só 22 e 80).

1.4 Criar o IP elástico

1. EC2 > **IPs elásticos** > **Alocar endereço IP elástico** > **Alocar**.
2. Marque o IP > Ações > **Associar endereço IP elástico** > instância `soufer-tools` > **Associar**.
3. Anote o IP. Na lista de instâncias, a coluna "IP elástico" deve mostrar esse endereço.

> **Correção.** Sem o IP elástico, o IP público muda toda vez que a instância é parada, e todo link e teste passa a apontar para um endereço que não existe mais. Faça este passo **agora**, não depois.

1.5 Conectar no servidor

Marque a instância > **Conectar** > aba **EC2 Instance Connect** > usuário `ubuntu` > **Conectar**. Para colar no terminal, use **Ctrl+Shift+V**.

Defina o seu IP uma vez (e **de novo** sempre que abrir uma aba nova do terminal):

```
export MEU_IP='SEU_IP_ELASTICO'
echo "IP: $MEU_IP"
```

Parte 2. Instalar e colocar o sistema no ar

2.1 Instalar os programas

```
sudo apt update && sudo apt -y upgrade
sudo apt -y install git nginx postgresql postgresql-contrib curl build-essential
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt -y install nodejs
sudo npm install -g pm2
node -v && nginx -v && psql --version && pm2 -v
```

Pronto quando: aparecem Node v22.x, nginx 1.x, psql e o número do PM2.

2.2 Variáveis, banco e projeto

Troque os valores pelos seus. A senha: só letras e números, nunca "root", "postgres", "admin", "123456" ou "password".

```
export SENHA_DB='UmaSenhaForte2026'
echo "IP: $MEU_IP | senha definida: ${#SENHA_DB} caracteres"
```

```
sudo -u postgres psql -c "CREATE USER soufer WITH PASSWORD '$SENHA_DB';"
sudo -u postgres psql -c "CREATE DATABASE soufer_prod OWNER soufer;"
for e in uuid-ossp unaccent pg_trgm; do
  sudo -u postgres psql -d soufer_prod -c "CREATE EXTENSION IF NOT EXISTS \"$e\";"
done
```

```
cd ~
git clone https://github.com/joaoaugusto-dev/PI-2026.2.git
cd PI-2026.2 && git checkout main
```

Pronto quando: aparecem `CREATE ROLE`, `CREATE DATABASE`, três `CREATE EXTENSION` e o clone do repositório.

2.3 Configurar, compilar e migrar a API

> **Correção.** O `echo` abaixo confere o IP antes de gravar o `.env`. Se `MEU_IP` estiver vazio, o `CORS_ORIGIN` ficaria `http://` (errado). E o arquivo já nasce com permissão 600.

```
echo "IP: $MEU_IP"   # não pode vir vazio
cd ~/PI-2026.2/api
JWT=$(openssl rand -hex 32)
cat > .env <<FIM
PORT=3000
NODE_ENV=production
CORS_ORIGIN=http://$MEU_IP
TRUST_PROXY_HOPS=1
DB_HOST=127.0.0.1
DB_PORT=5432
DB_NAME=soufer_prod
DB_USER=soufer
DB_PASSWORD=$SENHA_DB
DB_SSL=false
JWT_SECRET=$JWT
JWT_EXPIRES_IN=7d
JWT_CONSULTA_EXPIRES_IN=15m
FIM
chmod 600 .env
grep -E '^(CORS_ORIGIN|NODE_ENV|TRUST_PROXY_HOPS)=' .env
```

O último comando mostra só três variáveis (nunca use `cat .env`). Confira que `CORS_ORIGIN=http://SEU_IP_REAL`.

```
npm install
npm run build
npm run db:migrate
```

Pronto quando: o migrate termina com "Todas as migrations pendentes foram aplicadas com sucesso!" (15 migrations).

2.4 Ligar a API com o PM2

```
pm2 start npm --name soufer-api -- start
pm2 save
pm2 startup
```

O último comando imprime uma linha que começa com `sudo env PATH=...`. **Copie e rode essa linha**, depois rode `pm2 save` de novo. Sem ela a API não liga sozinha quando a EC2 reinicia.

```
pm2 status
curl http://127.0.0.1:3000/v1/health
```

Pronto quando: o PM2 mostra `online` no modo `fork` e o health responde `"status":"ok"` e `database connected`.

2.5 Compilar e publicar o front

```
cd ~/PI-2026.2/web
echo "VITE_API_URL=/v1" > .env
npm install
npm run build
sudo mkdir -p /var/www/soufer-web
sudo rm -rf /var/www/soufer-web/*
sudo cp -r dist/* /var/www/soufer-web/
sudo chmod -R o+rX /var/www /var/www/soufer-web
```

O `.env` do front precisa existir **antes** do build. Pronto quando: o build termina com "built in ...s" e `ls /var/www/soufer-web` mostra `index.html` e `assets`.

2.6 Nginx básico (a configuração de partida)

Montamos o arquivo em 4 blocos pequenos. Esta é a configuração **de partida**; as otimizações entram na Parte 5, depois de medir o "antes".

```
sudo rm -f /etc/nginx/sites-enabled/soufer /etc/nginx/sites-available/soufer
```

Bloco 1:

```
sudo tee /etc/nginx/sites-available/soufer >/dev/null <<'FIM'
server {
    listen 80 default_server;
    server_name _;
    client_max_body_size 12m;
    root /var/www/soufer-web;
    index index.html;
FIM
```

Bloco 2 (API):

```
sudo tee -a /etc/nginx/sites-available/soufer >/dev/null <<'FIM'
    location /v1/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
FIM
```

Bloco 3 (Swagger):

```
sudo tee -a /etc/nginx/sites-available/soufer >/dev/null <<'FIM'
    location /docs {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_hide_header Content-Security-Policy;
        proxy_hide_header Strict-Transport-Security;
    }
FIM
```

Bloco 4 (front e fechamento):

```
sudo tee -a /etc/nginx/sites-available/soufer >/dev/null <<'FIM'
    location / {
        try_files $uri /index.html;
    }
}
FIM
```

Ativar e testar:

```
sudo ln -sf /etc/nginx/sites-available/soufer /etc/nginx/sites-enabled/soufer
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl restart nginx
curl -I http://127.0.0.1/
curl http://127.0.0.1/v1/health
```

Pronto quando: aparecem "syntax is ok", "test is successful", `HTTP/1.1 200 OK` e o JSON do health. Se aparecer `unknown directive`, a colagem foi embaralhada: apague o arquivo e refaça os blocos.

2.7 Criar o seu administrador

> **Correção.** Não use as contas do seed (matrículas 0001, 0002 e 0053, senha pública `123456`) como administrador. Crie o seu agora, com uma matrícula de 4 dígitos que não seja dessas.

Digite o comando **em uma linha só**, sem colar junto com outros (a colagem dupla já criou um setor com nome errado):

```
cd ~/PI-2026.2/api
npm run db:admin -- 4271 "Seu Nome Completo" "Manutenção"
```

O comando imprime um link `http://SEU_IP/c/...`, de **uso único** e válido por 7 dias. Abra-o no navegador e defina a senha. O sistema usa **senha numérica de 6 dígitos**: escolha uma que não seja óbvia (nada de `123456`, `000000`, `654321`, datas). Você já entra logado. Se perder o link, rode o mesmo comando de novo. **Não mostre esse link em print antes de usá-lo.**

Se quiser conferir os setores: `export PGPASSWORD="$SENHA_DB"; psql -h 127.0.0.1 -U soufer -d soufer_prod -c "SELECT id, nome FROM setores;"`.

2.8 Dados de demonstração (opcional)

O banco nasce vazio. O seed básico cria setores, categorias, atividades e **três contas de teste com senha `123456`** (por isso a regra 8). O seed de demonstração cria 16 colaboradores, 36 ferramentas, 75 empréstimos e ocorrências.

```
cd ~/PI-2026.2/api
NODE_ENV=development npm run db:seed
cd ~/PI-2026.2
git fetch origin
git checkout origin/AWS -- api/db/seed-demo.sql
export PGPASSWORD="$SENHA_DB"
psql -h 127.0.0.1 -U soufer -d soufer_prod -v ON_ERROR_STOP=1 -f api/db/seed-demo.sql
```

Pronto quando: aparece o resumo com cerca de 36 ferramentas, 75 empréstimos e 3 ocorrências, e o dashboard abre cheio. **Logo depois**, siga o passo 6.5 para inativar as contas do seed. Para dados reais da Soufer, use a importação por CSV (Swagger em `/docs`, rota `POST /v1/importacoes/ferramentas`).

2.9 Validar tudo

Teste	Endereço	Esperado
Front	`http://SEU_IP/`	tela de login do SOUFER Tools
API	`http://SEU_IP/v1/health`	JSON com `"status":"ok"`
Swagger	`http://SEU_IP/docs`	lista de rotas
Admin	entre com a sua matrícula e a senha	dashboard
Banco	`psql -h 127.0.0.1 -U soufer -d soufer_prod -c "\dt"`	15 tabelas, dono `soufer`

Se o navegador der timeout (`ERR_CONNECTION_TIMED_OUT`), siga esta ordem: (1) `curl -s http://checkip.amazonaws.com` mostra o IP de hoje; abra exatamente esse; (2) porta 80 liberada no grupo; (3) grupo certo (`launch-wizard-1`, não `default`); (4) digite `http://` por extenso ou use janela anônima; (5) teste no celular em 4G.

Parte 3. Medir a linha de base ("antes")

Faça esta parte **antes** da Parte 5. Sem um "antes" não existe "melhora mensurável".

3.1 Recursos em repouso

```
date; uptime; free -m; df -h /; pm2 status
sudo ss -tlnp | grep -E ':(22|80|3000|5432)\b'
```

Print: `0.1 Recursos da instância em repouso.png`. Confira que o `5432` aparece só em `127.0.0.1`.

3.2 Cabeçalhos e tamanho do front (antes)

> **Correção.** Use o nome real do arquivo. Na primeira vez deixamos o marcador `ARQUIVO.js` no comando e o resultado foi a página inicial (HTML), não o JavaScript. O comando abaixo descobre o maior `.js` sozinho.

```
F=$(basename "$(ls -S /var/www/soufer-web/assets/*.js | head -1)"); echo "arquivo: $F"
curl -s -o /dev/null -w "sem gzip: %{size_download} bytes\n" http://127.0.0.1/assets/$F
curl -s -o /dev/null -H 'Accept-Encoding: gzip' -w "pedindo gzip: %{size_download} bytes\n"
http://127.0.0.1/assets/$F
curl -sI -H 'Accept-Encoding: gzip' http://127.0.0.1/assets/$F
curl -sI http://127.0.0.1/
```

No "antes" os dois tamanhos são iguais (o JavaScript não é comprimido) e não há `Cache-Control`. Anote o tamanho. Print: `0.2 Cabeçalhos e compressão do front (antes).png`.

3.3 Lighthouse (antes)

No **seu computador**, no Chrome, em **janela anônima** (Ctrl+Shift+N), abra `http://SEU_IP/` (com `http://` por extenso), aperte F12 > aba **Lighthouse** > marque *Performance* e *Best Practices*.

1. Rode em **Mobile** e depois em **Desktop**.
2. Anote Performance, FCP, LCP, TBT, CLS, Speed Index e os avisos (cache, JavaScript não usado, HTTPS).
3. Tire print de cada relatório (a métrica, os avisos e o rodapé com a data).

> **Correção.** Mantenha o **mesmo tamanho de janela** e as mesmas condições no "depois". Na nossa segunda rodada de desktop a janela estava mais estreita e a comparação ficou prejudicada.

Prints: `0.3 Lighthouse (antes)-mobile (n).png` e `0.3 Lighthouse (antes)-desktop (n).png`. O que obtivemos: celular 60 (FCP 6,1 s, LCP 7,2 s) e desktop 87 (FCP 1,4 s, LCP 1,8 s).

3.4 Teste de carga (antes)

> **Correção.** Use `ab -l`. O JSON do `/v1/health` muda de tamanho a cada chamada (hora e uptime) e, sem `-l`, o `ab` conta isso como "falha" (tipo *Length*). Rode **sem** `grep` ao menos uma vez e guarde o print completo.

```
sudo apt -y install apache2-utils
ab -l -n 1000 -c 20 http://127.0.0.1/ | grep -E 'Requests per second|Time per request|Failed'
ab -l -n 500 -c 10 http://127.0.0.1/v1/health | grep -E 'Requests per second|Time per
request|Failed'
```

Anote os `Requests per second`. Print: `0.4 Teste de carga leve (antes).png`. O que obtivemos: front 14.991 req/s (1,3 ms) e `/v1/health` cerca de 600 req/s (16 ms).

3.5 Tamanho do bundle (antes)

No **seu computador**, pasta `web`:

```
npm install
npm run build
npx vite-bundle-visualizer
```

O `build` imprime a lista de arquivos com tamanho e gzip (print). Se o `npx` perguntar `Ok to proceed? (y)`, responda `y`; o mapa abre no navegador (print). **Não commite** o `stats.html`. Na EC2, para ter os tamanhos do que está publicado:

```
cd /var/www/soufer-web/assets
ls -lS | head -15
for f in $(ls -S *.js | head -8); do echo "$f  bruto=$(wc -c < $f)  gzip=$(gzip -9c $f | wc -
c)"; done
cd ~
```

> **Correção.** Estes comandos são para a **pasta do front no seu PC** (ou para a pasta publicada na EC2). Rodar `npm install` na pasta `/home/ubuntu` dá erro de `package.json` não encontrado.

Prints: `0.5 Tamanho do bundle (antes) (n).png` e `2.5 Bundle do front (depois).png`.

3.6 Consulta do banco (plano de execução)

```
export PGPASSWORD="$(grep ^DB_PASSWORD= ~/PI-2026.2/api/.env | cut -d= -f2)"
psql -h 127.0.0.1 -U soufer -d soufer_prod -c "EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM
emprestimos ORDER BY data_retirada DESC, id DESC LIMIT 20;"
psql -h 127.0.0.1 -U soufer -d soufer_prod -c "\di+ idx_emprestimos*"
```

Pronto quando: o plano mostra `Index Scan using idx_emprestimos_data_retirada` e o `Execution Time`. O que obtivemos: 0,106 ms. Print: `0.6 Consulta do banco.png`.

Parte 4. Monitoramento (CloudWatch)

4.1 Papel do IAM para a instância

Console da AWS > EC2 > marque a instância > **Ações > Segurança > Modificar função do IAM** > escolha **`LabInstanceProfile`** > **Atualizar função do IAM**.

Confirme com o aviso verde ("LabInstanceProfile anexada à instância ... com êxito") ou, na EC2, com:

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-
ttl-seconds: 60")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-
data/iam/security-credentials/
```

Se o Learner Lab não aceitar o perfil, tire print da mensagem (é um desafio real para o relatório) e use o plano B: `free -m`, `df -h`, `pm2 monit` e `pm2 logs soufer-api` como evidência. Print: `1.2 Papel do IAM para a instância.png`.

4.2 Métricas nativas e monitoramento detalhado

EC2 > instância > aba **Monitoramento**. Se aparecer *Gerenciar monitoramento detalhado*, ative (intervalo de 1 minuto). Deixe a instância rodando pelo menos 15 a 30 minutos para os gráficos terem pontos. Print: `1.1 Métricas nativas da EC2.png`.

4.3 CloudWatch Agent: memória, disco e logs

```
cd /tmp
wget https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-
agent.deb
sudo dpkg -i -E ./amazon-cloudwatch-agent.deb
```

Crie a configuração (cole em blocos menores se o terminal embaralhar):

```
sudo tee /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json >/dev/null <<'FIM'
{
  "agent": { "metrics_collection_interval": 60, "run_as_user": "root" },
  "metrics": {
    "namespace": "CWAgent",
    "append_dimensions": { "InstanceId": "${aws:InstanceId}" },
    "metrics_collected": {
      "mem":  { "measurement": ["mem_used_percent"] },
      "disk": { "measurement": ["used_percent"], "resources": ["/"] }
    }
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          { "file_path": "/home/ubuntu/.pm2/logs/soufer-api-out.log",   "log_group_name":
"soufer-api",          "log_stream_name": "{instance_id}-out" },
          { "file_path": "/home/ubuntu/.pm2/logs/soufer-api-error.log", "log_group_name":
"soufer-api",          "log_stream_name": "{instance_id}-error" },
          { "file_path": "/var/log/nginx/access.log",                   "log_group_name":
"soufer-nginx-access", "log_stream_name": "{instance_id}" },
          { "file_path": "/var/log/nginx/error.log",                    "log_group_name":
"soufer-nginx-error",  "log_stream_name": "{instance_id}" }
        ]
      }
    }
  }
}
FIM
```

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -s
\
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status
```

Gere tráfego (abra o site e faça login) e espere 2 a 3 minutos. Se der erro de permissão: `sudo tail -30 /opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log` (um `AccessDenied` significa que o papel não permite publicar).

Pronto quando: o `status` mostra `"status": "running"` e `"configstatus": "configured"`; no console, **CloudWatch > Métricas > CWAgent** mostram `mem_used_percent` e `disk_used_percent` (dimensão com `path /`); e **Logs > Grupos de logs** mostram `soufer-api` e `soufer-nginx-access`.

O grupo `soufer-nginx-error` só aparece quando o arquivo de erros do Nginx tiver a primeira linha. Não é falha.

Prints: `1.3-A CloudWatch Agent memória, disco e logs (n).png` (instalação, configuração com o status `running`, métrica de memória, métrica de disco e os grupos de logs).

4.4 Consultar os logs (Logs Insights) e girar os logs do PM2

CloudWatch > **Logs Insights** > escolha o grupo `soufer-api` e cole **só isto** no editor:

```
fields @timestamp, @message
| sort @timestamp desc
| limit 20
```

Segunda consulta (erros):

```
fields @timestamp, @message
| filter @message like /error/
| sort @timestamp desc
| limit 20
```

> **Correção.** O editor do Logs Insights é só para consultas. Na primeira vez colamos junto os comandos `pm2 ...` na caixa de consulta e deu "Sem resultados". Os comandos do PM2 vão no **terminal da EC2**.

Sem ocorrências de "error" é bom sinal. Para um print mais forte, faça a mesma consulta no grupo `soufer-nginx-access` e filtre o `502` do teste do alarme (seção 4.8).

Rotação dos logs do PM2 (o disco é pequeno):

```
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
pm2 conf pm2-logrotate
```

Prints: `1.7 Logs consulta e rotação (n).png`.

4.5 Métrica de saúde da API (script no cron)

O CloudWatch não enxerga o `/v1/health` por fora. Um script a cada minuto chama o health pelo Nginx (passando por toda a cadeia) e envia a métrica `SouferTools/ApiHealth` com valor 1 (resposta 200) ou 0.

> **Correção.** O cron tem um `PATH` reduzido e pode não achar o `aws` instalado por snap. Use o caminho completo `/snap/bin/aws` no script e grave a saída do cron em um log para poder conferir.

```
sudo snap install aws-cli --classic
```

```
sudo tee /usr/local/bin/soufer-health.sh >/dev/null <<'FIM'
#!/bin/bash
CODE=$(curl -s -o /dev/null -m 5 -w "%{http_code}" http://127.0.0.1/v1/health)
if [ "$CODE" = "200" ]; then V=1; else V=0; fi
/snap/bin/aws cloudwatch put-metric-data --namespace SouferTools --metric-name ApiHealth --
value $V --region us-east-1
FIM
sudo chmod +x /usr/local/bin/soufer-health.sh
/usr/local/bin/soufer-health.sh && echo OK
```

```
( crontab -l 2>/dev/null; echo "* * * * * /usr/local/bin/soufer-health.sh >> /tmp/soufer-
health.log 2>&1" ) | crontab -
crontab -l
```

Confira que a linha do cron aparece **uma vez só**. Depois de 3 a 4 minutos:

```
cat /tmp/soufer-health.log
/snap/bin/aws cloudwatch get-metric-statistics --namespace SouferTools --metric-name ApiHealth
\
  --start-time $(date -u -d '15 minutes ago' +%FT%TZ) --end-time $(date -u +%FT%TZ) \
  --period 60 --statistics Minimum --region us-east-1
```

Pronto quando: o log está **vazio** (o `put-metric-data` não imprime nada quando dá certo; qualquer mensagem é erro) e o comando lista vários `Datapoints` com `"Minimum": 1.0`. No console, em **Métricas > SouferTools > ApiHealth**, use o intervalo de 1h (o primeiro ponto pode levar 1 a 2 minutos).

Prints: `1.4 Alarme de disponibilidade da API (n).png` (script e cron, gráfico, pontos mínimos).

4.6 Tópico do SNS e confirmação do e-mail

> **Correção.** Faça isto **antes** de criar o alarme. Se a assinatura estiver com "Confirmação pendente", o SNS não envia e-mail nenhum (foi o que aconteceu com o nosso primeiro disparo).

1. Console > **SNS** > Tópicos > **Criar tópico** (tipo Padrão), nome `soufer-alertas`.
2. No tópico, **Criar assinatura**: protocolo **E-mail**, endereço da equipe.
3. Abra o e-mail "AWS Notification - Subscription Confirmation" (olhe o spam) e clique em **Confirm subscription**.
4. Em SNS > Assinaturas, o status deve ser **Confirmado**. Se a mensagem não chegar, selecione a assinatura e use **Solicitar confirmação**.

Se o e-mail institucional filtrar a mensagem, tente um e-mail pessoal. Se o laboratório bloquear o SNS, tire print da mensagem e siga sem ação de e-mail (o alarme funciona do mesmo jeito).

4.7 Criar o alarme

CloudWatch > Alarmes > **Criar alarme** > **Selecionar métrica** > `SouferTools` > `Métricas sem dimensões` > `ApiHealth`.

Campo	Valor
Estatística	**Mínimo**
Período	**1 minuto**
Tipo de limite	Estático
Condição	**Menor que** `1`
Pontos de dados para o alarme	**2 de 2**
Dados ausentes	**Tratar como ruins (violando)**
Ação (estado Em alarme)	tópico `soufer-alertas`
Nome	`soufer-api-fora-do-ar`
Descrição	`API do SOUFER Tools sem responder em /v1/health`

"Dados ausentes como violando" faz o alarme disparar também se a métrica parar de chegar (por exemplo, se a máquina cair). Espere 2 a 3 minutos até o estado ficar **OK** (no começo aparece "Dados insuficientes").

Prints: etapa 1, configuração e alarme criado.

4.8 Testar o alarme de verdade

> **Correção.** Rode **um comando por vez** e espere. Na primeira vez colamos `stop`, `curl` e `start` juntos: a API ficou parada por menos de um segundo, e o alarme (que exige 2 minutos) não disparou.

1. Confirme o alarme em **OK** e tire print ("antes").
2. Pare a API e anote o horário:

```
date; pm2 stop soufer-api
```

3. Confirme o 502 (a API parada, o Nginx no ar):

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1/v1/health
```

4. **Espere cerca de 4 minutos** com a API parada. Atualize a página do alarme até virar **Em alarme** (no nosso teste, 3 min 47 s). Tire print com a hora (UTC) e confira o e-mail.
5. Religue e espere cerca de 3 minutos até voltar para **OK**:

```
date; pm2 start soufer-api
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1/v1/health
```

6. **Teste do e-mail por mudança forçada** (se o e-mail do disparo real não chegou, por exemplo porque a assinatura ainda estava pendente). O SNS só envia quando o estado **muda**, então force a mudança:

```
/snap/bin/aws cloudwatch set-alarm-state --alarm-name soufer-api-fora-do-ar --state-value OK --
state-reason "teste de envio" --region us-east-1
sleep 10
/snap/bin/aws cloudwatch set-alarm-state --alarm-name soufer-api-fora-do-ar --state-value ALARM
--state-reason "teste de envio do e-mail" --region us-east-1
```

O e-mail chega em 1 a 2 minutos. O estado forçado dura até a próxima avaliação (cerca de 1 minuto): se a API estiver rodando, volta para OK. No relatório, diga que o e-mail foi validado por estado forçado e mantenha o print do disparo real.

Prints: o alarme OK, o terminal com `date`, `stopped` e `502`, o alarme **Em alarme**, o e-mail recebido, o terminal com o `start`, o alarme OK de novo.

4.9 Painel (dashboard)

CloudWatch > **Painéis** > **Criar painel** `soufer-tools`. Para cada gráfico: **Adicionar widget > Linha > Métricas**, marque a métrica, dê o título e **Criar widget**. Salve o painel (botão **Salvar**, para não perder se a sessão expirar).

Título	Métrica
CPU (%)	EC2 > Por instância > `CPUUtilization`
Rede (bytes)	EC2 > Por instância > `NetworkIn` e `NetworkOut`
Memória (%)	CWAgent > `InstanceId` > `mem_used_percent`
Disco (%)	CWAgent > dimensão com `path /` > `disk_used_percent`
Saúde da API	SouferTools > Métricas sem dimensões > `ApiHealth` (estatística Mínimo, período 1 min)

Ajuste o período do painel para 1h ou 3h. Para o print, gere carga com `ab -l -n 3000 -c 20 http://127.0.0.1/` (a CPU e a rede sobem). Print: `1.6 Painel (dashboard).png`.

4.10 Extra (não feito por nós): alarmes de recursos

Pelo mesmo caminho do 4.7 (CloudWatch > Alarmes > Criar alarme), crie com ação no mesmo tópico:

- `CPUUtilization` maior que 70% (EC2 > Por instância, 5 minutos, 2 de 2).
- `StatusCheckFailed` maior ou igual a 1 (1 minuto).
- `mem_used_percent` maior que 80% (CWAgent).
- `disk_used_percent` maior que 80% (CWAgent, `path /`), importante se o disco for de 8 GiB.

Parte 5. Otimização

5.1 Nginx: compressão, cache, cabeçalhos e limite de requisições

Antes de reescrever, guarde a versão atual e confira o gzip global (no Ubuntu o `gzip on;` já vem ativo; **não repita** `gzip on` no site):

```
grep -n gzip /etc/nginx/nginx.conf
sudo cp /etc/nginx/sites-available/soufer ~/soufer.nginx.bak
```

Bloco 0, limite de requisições (a zona fica fora do `server`):

```
sudo tee /etc/nginx/conf.d/soufer-limites.conf >/dev/null <<'FIM'
limit_req_zone $binary_remote_addr zone=soufer_api:10m rate=20r/s;
FIM
```

Bloco 1, cabeçalho do site com compressão e cabeçalhos de segurança:

```
sudo tee /etc/nginx/sites-available/soufer >/dev/null <<'FIM'
server {
    listen 80 default_server;
    server_name _;
    server_tokens off;
    client_max_body_size 12m;
    root /var/www/soufer-web;
    index index.html;

    gzip_comp_level 5;
    gzip_min_length 1024;
    gzip_vary on;
    gzip_proxied any;
    gzip_types text/css application/javascript application/json image/svg+xml text/plain;

    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
FIM
```

Bloco 2, API com limite geral:

```
sudo tee -a /etc/nginx/sites-available/soufer >/dev/null <<'FIM'
    location /v1/ {
        limit_req zone=soufer_api burst=40 nodelay;
        limit_req_status 429;
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
FIM
```

Bloco 3, Swagger:

```
sudo tee -a /etc/nginx/sites-available/soufer >/dev/null <<'FIM'
    location /docs {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_hide_header Content-Security-Policy;
        proxy_hide_header Strict-Transport-Security;
    }
FIM
```

Bloco 4, front com cache e fechamento:

```
sudo tee -a /etc/nginx/sites-available/soufer >/dev/null <<'FIM'
    location /assets/ {
        expires 1y;
        try_files $uri =404;
    }

    location / {
        expires -1;
        try_files $uri /index.html;
    }
}
FIM
```

```
sudo nginx -t && sudo systemctl reload nginx
```

O aviso amarelo sobre `systemctl daemon-reload` é inofensivo. Se o `nginx -t` falhar, restaure com `sudo cp ~/soufer.nginx.bak /etc/nginx/sites-available/soufer` e refaça bloco a bloco.

O que cada parte faz: `expires 1y` em `/assets/` é seguro porque o Vite coloca um hash no nome dos arquivos (se o conteúdo muda, o nome muda); `expires -1` no `index.html` faz o navegador sempre buscar a versão nova depois de um deploy; `server_tokens off` esconde a versão do Nginx; `limit_req` aplica 20 requisições por segundo por IP, com uma tolerância (burst) de 40, e devolve 429 acima disso.

Confirme (use o mesmo bloco do 3.2):

```
F=$(basename "$(ls -S /var/www/soufer-web/assets/*.js | head -1)"); echo "arquivo: $F"
curl -s -o /dev/null -w "sem gzip: %{size_download} bytes\n" http://127.0.0.1/assets/$F
curl -s -o /dev/null -H 'Accept-Encoding: gzip' -w "com gzip: %{size_download} bytes\n"
http://127.0.0.1/assets/$F
curl -sI -H 'Accept-Encoding: gzip' http://127.0.0.1/assets/$F
curl -sI http://127.0.0.1/
```

Pronto quando: os assets respondem com `Content-Encoding: gzip`, `Vary: Accept-Encoding`, `Cache-Control: max-age=31536000` e `Expires` daqui a 1 ano; o `/` responde com `Cache-Control: no-cache`; o `Server` aparece sem versão. O que obtivemos: 461.552 para 141.406 bytes (−69,4%).

Prints: `2.1 Nginx compressão, cache e cabeçalhos (front) (n).png` (conferência do gzip, cada bloco, o `nginx -t`, os cabeçalhos).

> **Ajuste opcional (ainda não testado por nós).** Alguns cabeçalhos de segurança saem em dobro nas rotas `/v1/`, uma vez do Helmet e outra do Nginx (a `Referrer-Policy` aparece com dois valores). É inofensivo. Para tirar a duplicidade, mova os três `add_header` do nível do `server` para dentro de `location /assets/` e de `location /` (o `add_header` dentro de um `location` substitui os do nível acima; o `expires` não é afetado), e teste de novo com `nginx -t` e `curl -sI`.

5.2 Limite de requisições: prova

```
for i in $(seq 1 120); do curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1/v1/health;
done | sort | uniq -c
```

Pronto quando: aparecem respostas `200` e `429` (no nosso teste, 60 de cada). O limite geral fica por cima dos que a API já tem em login, consulta e convite. Print: `2.2 Rate limit geral.png`.

> **Atenção.** Depois do limite, **não use `ab` em `/v1/health`**: ele devolve 429 e o resultado deixa de ser comparável com o "antes". Meça só o front (`/`).

5.3 PM2: limite de memória e logs

A API deve ficar em **1 instância, modo fork**: o limite de requisições guarda os contadores na memória do processo, e em modo cluster cada cópia teria o seu contador.

```
pm2 restart soufer-api --max-memory-restart 400M --update-env
pm2 save
pm2 describe soufer-api | grep -E 'exec mode|restarts|uptime|memory'
```

Pronto quando: aparecem `fork_mode` e `max memory restart 419430400` (400 MB). Print: `2.4 Ajustes de PM2 e PostgreSQL (1).png`.

5.4 PostgreSQL: parâmetros para 4 GiB de memória

Avise a equipe: o reinício derruba o banco por alguns segundos.

```
sudo -u postgres psql -c "ALTER SYSTEM SET shared_buffers='1GB';"
sudo -u postgres psql -c "ALTER SYSTEM SET effective_cache_size='2GB';"
sudo -u postgres psql -c "ALTER SYSTEM SET work_mem='8MB';"
sudo -u postgres psql -c "ALTER SYSTEM SET maintenance_work_mem='128MB';"
sudo systemctl restart postgresql
sudo -u postgres psql -c "SHOW shared_buffers;"
```

> **Correção.** Espere a API subir antes do `curl`. Na primeira vez ele rodou no mesmo instante do `pm2 restart` e voltou vazio.

```
pm2 restart soufer-api
sleep 5
curl -s http://127.0.0.1:3000/v1/health
sudo -u postgres psql -c "SHOW effective_cache_size;" -c "SHOW work_mem;" -c "SHOW
maintenance_work_mem;"
pm2 save
```

Pronto quando: `shared_buffers` mostra `1GB` (só muda com reinício), os outros mostram `2GB`, `8MB` e `128MB`, e o health volta com `"status":"ok"` e `connected`. Prints: `2.4 Ajustes de PM2 e PostgreSQL (2) e (3).png`.

5.5 Medir o "depois"

Repita no PC o Lighthouse do 3.3 (**mesma janela e mesmas condições**; mobile e desktop) e os comandos do 3.2. Para o teste de carga use só o front:

```
ab -l -n 1000 -c 20 http://127.0.0.1/ | grep -E 'Requests per second|Time per request|Failed'
```

Prints: `2.3 Lighthouse e carga (depois) mobile (n).png` e `desktop (n).png`. O que obtivemos no celular: nota 60 para 84, FCP 6,1 s para 2,9 s e LCP 7,2 s para 3,6 s; o aviso de cache (1.173 KiB) sumiu e o JavaScript não usado caiu de 405 KiB para 99 KiB.

A nota de Boas Práticas continua em 78 porque o sistema ainda é HTTP (seção 8.1).

5.6 Bundle: análise (decisão registrada)

O front já usa divisão por rota (um arquivo por página). Os maiores arquivos que medimos foram o `index` (461.552 bytes, 139.938 com gzip -9) e o da tela de Análises (378.385 bytes, com a biblioteca de gráficos, que só é baixado nessa tela). Não mudamos o código nesta etapa; o ganho veio da compressão e do cache. Registre a análise (mapa do visualizer e a listagem) como evidência.

Parte 6. Segurança

6.1 Cabeçalhos do Helmet e CORS

O Helmet é um módulo da API que adiciona cabeçalhos de segurança. O CORS define quais sites podem chamar a API pelo navegador (o navegador envia `Origin` e só entrega a resposta se a API devolver `Access-Control-Allow-Origin` para ela).

```
grep -E '^(CORS_ORIGIN|NODE_ENV|TRUST_PROXY_HOPS)=' ~/PI-2026.2/api/.env
curl -sI http://127.0.0.1/v1/health
echo "--- origem invasora (NÃO deve ter Access-Control-Allow-Origin):"
curl -si -H "Origin: http://site-invasor.example" http://127.0.0.1/v1/health | grep -iE
"^HTTP|access-control|^vary"
echo "--- origem permitida (deve ter Access-Control-Allow-Origin igual ao IP):"
curl -si -H "Origin: http://$MEU_IP" http://127.0.0.1/v1/health | grep -iE "^HTTP|access-
control|^vary"
```

> **Correção.** Não use `| head -15` no teste do CORS: os cabeçalhos do Helmet ocupam as primeiras linhas e o corte esconde os de CORS. Use o `grep` acima.

Pronto quando: o `.env` mostra `CORS_ORIGIN=http://SEU_IP_REAL` (nunca `*`), `NODE_ENV=production` e `TRUST_PROXY_HOPS=1`; a resposta traz os cabeçalhos do Helmet (`Content-Security-Policy`, `Strict-Transport-Security`, `X-Frame-Options`...); a origem invasora **não** recebe `Access-Control-Allow-Origin`; a origem do IP recebe.

Se a origem permitida **também** não receber, o `CORS_ORIGIN` está errado (no nosso caso estava `http://IP_NOVO`, o texto do tutorial). Corrija:

```
sed -i "s#^CORS_ORIGIN=.*#CORS_ORIGIN=http://$MEU_IP#" ~/PI-2026.2/api/.env
grep -E '^(CORS_ORIGIN|NODE_ENV|TRUST_PROXY_HOPS)=' ~/PI-2026.2/api/.env
pm2 restart soufer-api --update-env
pm2 save
sleep 5
```

e repita o teste. Prints: `3.1 Cabeçalhos da API e CORS (n).png` (Helmet, o erro, a correção e a prova).

6.2 Permissão do `.env`

```
chmod 600 ~/PI-2026.2/api/.env && ls -l ~/PI-2026.2/api/.env
```

Pronto quando: `-rw-------` e dono `ubuntu`. Print só dessa saída: `3.2 Permissão do .env.png`. Nunca o conteúdo.

6.3 Firewall

Reconfira o grupo de segurança (passo 1.3): só 22 e 80, nenhuma regra para o banco. O banco também só escuta em `127.0.0.1`. **Só no fim de tudo** (depois de todas as evidências), troque a origem da porta 22 para **Meu IP**. Isso pode quebrar o botão *Conectar* do navegador (EC2 Instance Connect), por isso anote a data e deixe por último.

6.4 fail2ban e atualizações automáticas

```
sudo apt -y install fail2ban unattended-upgrades
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
systemctl is-enabled unattended-upgrades
```

Pronto quando: aparece `Status for the jail: sshd` (com `Currently failed` e `Currently banned`) e o `unattended-upgrades enabled`. Os avisos `SyntaxWarning: invalid escape sequence` da instalação são de arquivos de teste do pacote e podem ser ignorados. Prints: `3.4 Proteções do sistema (n).png`.

6.5 Contas padrão do seed e administrador

Se você rodou o seed (passo 2.8), as contas **0001, 0002 e 0053** têm a senha pública `123456`.

1. No navegador, tente entrar com cada uma (senha `123456`). Se a mensagem for "Login realizado com sucesso!", a conta está aberta (tire print: é o achado do relatório).

2. Entre com o **seu** administrador (passo 2.7). **Se o seu admin for uma das contas do seed, não a inative antes de criar outro**: crie um novo com `db:admin` e uma matrícula de 4 dígitos diferente, defina a senha pelo link e entre com ele.
3. Em **Colaboradores**, inative **0001, 0002 e 0053**. Os dados ficam no banco (inativar não apaga).
4. Saia e tente entrar de novo com cada uma, senha `123456`. Deve aparecer **"Usuário inativo. Contate o administrador."** Tire print de cada uma. Entre com o seu administrador para confirmar que o seu acesso continua.

Se precisar de uma conta de demonstração para a apresentação, crie uma nova pelo painel admin e gere o link de acesso: não reative as do seed.

> **Atenção com os prints.** O link do `db:admin` e a tela de definição de senha mostram o token na URL. Ele só vale uma vez: depois que a senha é definida o link morre. Antes disso, não mostre. Se for guardar esses prints, confirme que o link já foi usado.

Prints: `3.5 Contas do seed e vulnerabilidades de dependências (n).png`.

6.6 Dependências com vulnerabilidades (npm audit)

No **seu computador**, na branch `AWS`. **Não use `--force`**: ele aceitaria mudanças incompatíveis.

```
cd api
npm audit --omit=dev
npm audit fix
npm test
npm audit --omit=dev
cd ..\web
npm audit --omit=dev
npm audit fix
npm run build
npm test
npm audit --omit=dev
```

Resultados que tivemos: API de 9 achados (6 moderados, 3 altos) para 5 (3 moderados, 2 altos, ligados ao `glob`, `minimatch` e `zod`); front de 3 achados (1 moderado, 2 altos) para **0** em produção. O `audit fix` do front ainda lista 6 itens em ferramentas de desenvolvimento, que não vão para o pacote publicado. O `zod` (alto) só se corrige com atualização incompatível: registre como risco aceito, mitigado em parte pelo limite de corpo (10 MB na API, 12 MB no Nginx) e pelo rate limit. Se quiser tentar, rode `npm install zod@4.6.7` e `npm test`: se todos os testes passarem, mantenha; se der conflito de dependência ou algum teste falhar, desfaça com `git checkout -- package.json package-lock.json`.

Pronto quando: todos os testes passam (nós: 777 na API, 47 arquivos; 72 no front, 11 arquivos) e o build do front compila. Só os arquivos `package-lock.json` mudam (confira com `git status`). Para commitar:

```
git add api/package-lock.json web/package-lock.json
git commit -m "chore(deps): aplicar npm audit fix nas dependências da api e do web"
```

Esse trabalho acontece só no seu computador: **não vai para a EC2** a menos que você publique o código de novo (não é necessário para a entrega). Prints: `npm audit` antes e depois, e os testes passando.

Parte 7. Backup, restauração e snapshot

7.1 Backup diário do banco

> **Correção.** Coloque o `umask 077` no próprio script desde o início, e **proteja o diretório antes de copiar qualquer coisa**: assim o diretório sai com permissão 700 e o dump com 600. (Na primeira vez criamos o dump legível por todos e tivemos de corrigir depois.)

```
sudo tee /usr/local/bin/soufer-backup.sh >/dev/null <<'FIM'
#!/bin/bash
set -e
umask 077
DIR=/var/backups/soufer
mkdir -p "$DIR"
export PGPASSWORD="$(grep ^DB_PASSWORD= /home/ubuntu/PI-2026.2/api/.env | cut -d= -f2)"
pg_dump -h 127.0.0.1 -U soufer -d soufer_prod -Fc -f "$DIR/soufer_prod_$(date +%F_%H%M).dump"
find "$DIR" -name '*.dump' -mtime +7 -delete
FIM
sudo chmod +x /usr/local/bin/soufer-backup.sh
sudo /usr/local/bin/soufer-backup.sh && sudo ls -lh /var/backups/soufer
```

```
( sudo crontab -l 2>/dev/null; echo "0 3 * * * /usr/local/bin/soufer-backup.sh" ) | sudo
crontab -
sudo crontab -l
sudo ls -ld /var/backups/soufer
sudo ls -l /var/backups/soufer
```

O backup roda às **03:00 UTC (meia-noite em Brasília)** e guarda 7 dias. Pronto quando: aparece um arquivo `soufer_prod_AAAA-MM-DD_HHMM.dump`, o cron mostra `0 3 * * *`, o diretório está `drwx------` e o dump `-rw-------`.

Se o dump anterior foi criado sem o `umask` (legível por todos), proteja assim:

```
sudo chmod 700 /var/backups/soufer
sudo sh -c 'chmod 600 /var/backups/soufer/*.dump'
```

> **Correção.** Use `sudo sh -c '...'` quando houver `*`. Depois do `chmod 700`, o shell do usuário comum não lê mais a pasta e **não expande** o `*`: o `sudo chmod 600 /var/backups/soufer/*.dump` dá "No such file or directory".

Prints: `4.1 Backup diário do banco (n).png`.

7.2 Teste de restauração (obrigatório)

Backup sem teste de restauração não prova nada. Como o diretório é restrito, o root lê o arquivo e passa para o `pg_restore` por entrada padrão:

```
ULTIMO=$(sudo sh -c 'ls -t /var/backups/soufer/*.dump | head -1'); echo "$ULTIMO"
sudo -u postgres createdb soufer_teste_restore
sudo cat "$ULTIMO" | sudo -u postgres pg_restore -d soufer_teste_restore
sudo -u postgres psql -d soufer_teste_restore -c "\dt" | tail -8
sudo -u postgres psql -d soufer_teste_restore -c "SELECT count(*) AS ferramentas FROM
ferramentas;"
sudo -u postgres dropdb soufer_teste_restore
```

Pronto quando: aparecem as tabelas (15, dono `soufer`) e a contagem de ferramentas igual à do sistema (no nosso caso, 36). O `dropdb` apaga só a cópia de teste; a produção não é tocada. Print: `4.2 Teste de restauração.png`.

7.3 Snapshot do disco

1. Console > EC2 > **Instâncias** > `soufer-tools` > aba **Armazenamento**: anote o **ID do volume** raiz.
2. EC2 > **Volumes**: marque **esse** volume (a conta pode ter mais de um) > **Ações > Criar snapshot**, descrição `soufer-tools-AAAA-MM-DD`.
3. EC2 > **Snapshots**: espere o estado **Concluído**, 100%. Tire print da lista e dos detalhes (volume de origem, tamanho, horário).

> **Correção.** Confira que o volume do snapshot é o da instância (passo 1). Na nossa conta havia dois volumes de 8 GiB e não confirmamos qual era o da `soufer-tools`.

O erro "Falha ao obter o status da política padrão" do Data Lifecycle Manager é uma limitação do laboratório: o snapshot é manual. Prints: `4.3 Snapshot do disco (extra) (n).png`.

Parte 8. Extras recomendados (não feitos por nós)

8.1 HTTPS

O laboratório não dá domínio, então o sistema fica em HTTP (o Lighthouse avisa "Does not use HTTPS"). Para ter HTTPS de graça:

1. Crie um subdomínio gratuito em duckdns.org apontando para o IP elástico.
2. No grupo de segurança, libere **HTTPS (443)**.
3. Na EC2 (troque o domínio):

```
sudo sed -i 's/server_name _;/server_name soufer.duckdns.org;/' /etc/nginx/sites-
available/soufer
sudo apt -y install certbot python3-certbot-nginx
sudo certbot --nginx -d soufer.duckdns.org
sudo sed -i "s#^CORS_ORIGIN=.*#CORS_ORIGIN=https://soufer.duckdns.org#" ~/PI-2026.2/api/.env
pm2 restart soufer-api --update-env
pm2 save
```

Teste `https://soufer.duckdns.org/v1/health`. Se não fizer, registre no relatório: "HTTPS não adotado por falta de domínio; risco: tráfego em texto claro; mitigação: rate limit, bloqueio por tentativas no login, CORS restrito e Helmet; próximo passo: domínio com Certbot ou ACM".

8.2 Provas de reinício

Processo:

```
pm2 describe soufer-api | grep -E 'pid|restarts'
kill -9 PID_MOSTRADO
sleep 3
pm2 status
```

Máquina (derruba o sistema por 1 a 2 minutos):

```
sudo reboot
```

Reconecte e rode `uptime; pm2 status; curl -s http://127.0.0.1/v1/health; systemctl is-active nginx postgresql`. Tudo deve voltar sozinho (comprova o `pm2 startup` e o `pm2 save`).

8.3 Outras melhorias

- Restringir a porta 22 ao IP da equipe (seção 6.3), por último.
- Amarrar a API a `127.0.0.1` (hoje ela escuta em todas as interfaces e só o grupo de segurança a protege). Verifique se a API aceita uma variável de ambiente de endereço antes de mexer.
- Copiar os backups para fora da instância (por exemplo, um bucket S3).
- Os alarmes de recursos da seção 4.10.

Parte 9. Evidências e relatório

9.1 Quadro de evidências

#	Evidência	Passo
1	Recursos em repouso e portas	3.1
2	Cabeçalhos e tamanho do front, antes e depois	3.2 e 5.1
3	Lighthouse (celular e desktop), antes e depois	3.3 e 5.5

#	Evidência	Passo
4	Teste de carga, antes (e depois só do front)	3.4 e 5.5
5	Mapa e listagem do bundle	3.5
6	`EXPLAIN` do banco	3.6
7	Gráficos nativos da EC2 e o papel do IAM	4.1 e 4.2
8	Agente em execução, memória, disco e grupos de logs	4.3
9	Logs Insights e rotação	4.4
10	Métrica de saúde (script, gráfico, pontos)	4.5
11	SNS (assinatura confirmada) e criação do alarme	4.6 e 4.7
12	Teste do alarme (OK, parada, 502, Em alarme, e-mail, volta ao OK)	4.8
13	Painel com os cinco gráficos	4.9
14	Nginx: cada bloco, `nginx -t` e os cabeçalhos	5.1
15	Rate limit (200 e 429)	5.2
16	PM2 e parâmetros do PostgreSQL	5.3 e 5.4
17	Helmet, CORS (inclusive o erro e a correção)	6.1
18	`.env` com permissão 600, grupo de segurança e fail2ban	6.2 a 6.4
19	Contas do seed e o novo administrador	6.5
20	`npm audit` antes e depois, e testes passando	6.6
21	Backup, cron, permissões e restauração	7.1 e 7.2
22	Snapshot	7.3

9.2 Onde guardar e como nomear

Guarde tudo em `docs/entregas-pi/computacao-em-nuvem-rodrigo-marudi/prints-aws/`, com o número do passo no nome. As notas de medição vão em `docs/entregas-pi/computacao-em-nuvem-rodrigo-marudi/10-11-2026/` (por exemplo, `00-baseline-antes-carga-ab.md`).

9.3 O relatório (PDF)

O relatório segue o padrão do PDF de 13/10/2026: capa institucional, seções numeradas, figuras legendadas ("Figura N – ...") e número de página. Estrutura que usamos:

1. Contexto
2. Ambiente e linha de base (diagrama, especificações, medições "antes")
3. Ajustes na infraestrutura
4. Otimizações implementadas (com a tabela antes e depois)
5. Monitoramento dos recursos
6. Segurança e estabilidade
7. Principais desafios encontrados na nuvem

8. O que fizemos além do pedido
9. Pendências e ressalvas (dizer com clareza o que ficou de fora)
10. Resultado
11. Anexos (Nginx final, scripts de saúde e backup, configuração do agente, índice das evidências)

Explique cada conceito antes do resultado (CloudWatch, SNS, gzip, cache, rate limit, Helmet, CORS, `EXPLAIN`, backup e snapshot), cite números reais, e **só afirme o que há evidência**. Não esconda os erros: eles viram a seção de desafios.

9.4 Git: sem acidentes

- Trabalhe na branch `AWS`. Não faça PR para a `main` nem para a `development` sem combinar com a equipe.
- Use `git add` **sempre por caminho exato** (nunca `git add .` nem pasta inteira), para os prints soltos e arquivos temporários não entrarem no commit.
- Commits no padrão Conventional Commits, por etapa, citando a issue (por exemplo `docs(infra): evidências de monitoramento CloudWatch e teste do alarme (#75)`).
- Antes de commitar prints, confira que nenhum mostra conteúdo do `.env`, `JWT_SECRET`, senha do banco ou link de convite ainda válido.

Parte 10. Solução de problemas

Sintoma	Causa provável	Correção
Página "Criar modelo de execução"	Tela de Launch Template	Cancelar; usar Instâncias > Executar instâncias
`apt: command not found`	AMI Amazon Linux	Refazer com Ubuntu
A API recusa subir com "FATAL: ... JWT_SECRET"	`NODE_ENV` vazio, ou senha/segredo fraco ou de exemplo	`NODE_ENV=production` e senha forte (só letras e números)
`unknown directive` ou "cation" no Nginx	Colagem embaralhada no terminal	Apagar o arquivo e refazer em blocos pequenos
Timeout ao abrir o site	IP mudou, porta 80 fechada, grupo errado, https, rede	Checklist do passo 2.9
`ERR_CONNECTION_TIMED_OUT` só para outras pessoas	IP temporário (sem IP elástico) ou lab desligado	Criar o IP elástico; religar a instância
`command not found` ao rodar `db:seed`	Digitado fora da pasta `api` ou sem `npm run`	`cd ~/PI-2026.2/api` e `npm run db:seed`
"Já existem ferramentas no banco" no `seed-demo.sql`	Trava de segurança (só roda em banco sem ferramentas)	Nada a fazer

Sintoma	Causa provável	Correção
Gráfico `ApiHealth` vazio	O cron ainda não completou o primeiro ciclo	Esperar 3 a 4 minutos; conferir `cat /tmp/soufer-health.log`
Log do cron com `command not found`	PATH reduzido do cron	Usar `/snap/bin/aws` no script
Alarme não dispara no teste	API parada por menos de 2 minutos	Parar, esperar 4 minutos e só então religar
E-mail do alarme não chega	Assinatura do SNS pendente, spam ou filtro	Confirmar (SNS > Assinaturas); testar com `set-alarm-state`
Origem permitida sem `Access-Control-Allow-Origin`	`CORS_ORIGIN` com texto de exemplo	`sed` do passo 6.1 e `pm2 restart --update-env`
`chmod ... *.dump`: "No such file or directory"	`*` expandido pelo usuário sem acesso à pasta	`sudo sh -c 'chmod 600 /var/backups/soufer/*.dump'`
`curl` do health vazio logo após `pm2 restart`	A API ainda estava subindo	`sleep 5` antes do `curl`
`ab` mostra falhas do tipo Length	JSON do health muda de tamanho	Usar `ab -l`
`ab` em `/v1/health` devolve muitos erros depois do 5.1	O `limit_req` responde 429	Medir só o front (`/`)
Logs Insights sem resultados	Comandos do terminal colados no editor, ou grupo sem a palavra	Só a consulta no editor; os comandos no terminal
`npm install` em `/home/ubuntu` falha	Pasta errada (sem `package.json`)	Rodar na pasta `web` ou `api` do projeto
Disco quase cheio	Volume de 8 GiB, logs e backups	Disco de 20 GiB; `pm2-logrotate`; alarme de disco; retenção de 7 dias
Learner Lab bloqueia o IAM, o SNS ou o agente	Limitação do laboratório	Tirar print da mensagem (é desafio) e usar as alternativas dos passos

---

Checklist final de entrega

- [ ] Instância Ubuntu com 20 GiB, IP elástico e firewall só com 22 e 80
- [ ] Sistema no ar: front, `/v1/health`, `/docs` e banco com 15 tabelas
- [ ] "Antes" medido (recursos, cabeçalhos, Lighthouse, carga, bundle, `EXPLAIN`)
- [ ] Agente do CloudWatch rodando; métricas de memória e disco; grupos de logs
- [ ] Métrica de saúde da API chegando; assinatura do SNS **confirmada**
- [ ] Alarme `soufer-api-fora-do-ar` testado de verdade (parada, 502, Em alarme, e-mail, OK)
- [ ] Painel `soufer-tools` com os cinco gráficos
- [ ] Nginx otimizado (gzip, cache, cabeçalhos, limite) e provado com `curl` e Lighthouse
- [ ] PM2 em fork com limite de memória; PostgreSQL ajustado; "depois" medido

- [ ] CORS correto (origem invasora bloqueada, origem do IP aceita); `.env` em 600; fail2ban ativo
- [ ] Contas do seed inativas e administrador próprio entrando
- [ ] `npm audit fix` (sem `--force`), testes passando e commit só dos arquivos de lock
- [ ] Backup diário protegido, restauração testada e snapshot concluído
- [ ] Prints com data e hora, sem segredos; nomes com o número do passo
- [ ] Relatório em PDF no padrão, com as pendências ditas com clareza